

A new Python-based tool for simulation of dynamical systems

Peter Corke

Summary

This article introduces **bdsim** – a new, and free, software tool that computes the time response of continuous- and discrete-time dynamical systems. Dynamical systems are conceptualized in block diagram form, but represented directly in concise Python code. Block diagrams are: a powerful and intuitive way to represent systems; are foundational in undergraduate control courses and textbooks; and widely used in engineering practice. Python is a very popular open-source programming language that is increasingly used in engineering education, and it is the language of choice for robotics and AI development – Python skills are highly sought after by employers. For educational use, **bdsim** can dramatically reduce the total cost of provisioning simulation capability compared to proprietary tools, while also avoiding the inconvenience of voluminous local software installs and licence manager complexity.

The various constituent blocks of the system are object instances, as is the block diagram itself. Methods of the latter are used to add blocks and connect their input and output ports. These connections can transmit *any* Python type including vectors and multi-dimensional arrays. Over 70 blocks have been implemented: various linear and nonlinear functions, signal sources, graphical display blocks, as well as continuous- and discrete-time dynamics – it is straightforward to define new blocks. The block diagram can be run

and its time response displayed graphically in scope blocks, while also being available for further analysis.

INTRODUCTION

bdsim (block diagram simulator) is a new Python-based tool for simulating continuous- and/or discrete-time dynamical systems. Systems are conceptualized in block diagram form, but represented directly in Python code where each block is an object instance. **bdsim** can simulate small-scale *textbook* systems for pedagogical purposes, but it is also capable of modeling large and complex systems of systems such as the one shown in Fig. 1. **bdsim** is an open-sourced (MIT Licence) Python 3 package [1] available on PyPI [2].

Block diagrams, see for example Fig. 1 and 2, are graphical representations of systems. They are foundational in undergraduate control courses and textbooks, for example [4]. They are also widely used in engineering practice to provide intuitive abstractions of complex systems of systems, and serve as a common language for engineers to communicate system design and functionality.

In a block diagram, the system's components are represented as blocks, each of which must have at least one port comprising zero or more input and output ports. The values at the output ports are a function of the values at the input ports, the block's internal state variables if any, and time. Output ports are connected to one or more input ports, and the connections are often denoted by directed line segments. In a real system these connections might be implemented by wires (the metaphor used in **bdsim**) or by a network connection. Blocks can be static

Digital Object Identifier 10.1109/MCS.2020.0000000
Date of current version: XXXXXX

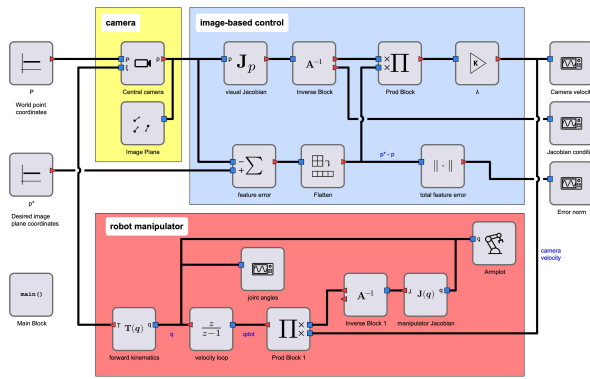


Figure 1 Block diagram model drawn and displayed using `bdedit` and simulated using `bdsim`. The model depicts an image-based visual servoing system where features from an eye-in-hand camera drive the robot hand to a desired pose with respect to the target. This model, and simulation results, are discussed in [3], Sec. 16.4.1. (Used with permission.) Input ports are denoted by small blue squares and output ports are denoted by small red triangles pointing in the direction of signal flow.

(linear or nonlinear functions) or dynamic (continuous- or discrete-time dynamical systems). For large systems, the block diagram can be hierarchical where blocks contain other block diagrams. There is no standard for the graphical representation of block diagrams, but there is a rough consensus across the common control textbooks and software packages such as Simulink®. One disadvantage of block diagram is that, for any system, they are not unique – different engineers may draw different block diagrams for the same system.

Dynamic systems modeling and control design is, today, dominated by proprietary software tools but it was not always this way. During the early 1980s there was an explosion of ideas and tools, from academia and industry, in the space of what was then called Computer Aided Control System Design (CACSD). One early and notable tool was Simnon, from the University of Lund [5], which was essentially a custom programming language for describing dynamic systems – a system-modeling language. For example a van der Pol oscillator in chained form could be expressed as shown in Listing 1. Lines 3–4 declare the state variables and their derivatives, lines 5–6 compute the state derivatives from the current state values, and line 7 defines a constant. While powerful with respect to dynamic systems, the language is quite limited with respect to the other features we expect of a programming language.

In 1996 the author of Simnon created the Modelica modeling language [7], [8] and the van der Pol oscillator model, expressed in Modelica, is shown in Listing 2. There are very obvious similarities with Listing 1 – lines 2–3 define the state variables and their initial values, line

```
1 continuous system VDPOL
2 "The van der Pol equation
3 state x y
4 der dx dy
5 dy = x
6 dx = lambda * x * (1 - y*y) - y
7 lambda: 0.3
8 END
```

Listing 1 Simnon model of a van der Pol oscillator. [5]

```
1 model VanDerPol "Van der Pol oscillator
  model"
2   Real x(start = 1,fixed=true);
3   Real y(start = 1,fixed=true);
4   parameter Real lambda = 0.3;
5   equation
6     der(x) = y;
7     der(y) = -x + lambda*(1 - x*x)*y;
8 end VanDerPol;
```

Listing 2 Modelica model of a van der Pol oscillator. [6]

4 defines a parameter, and lines 6–7 compute the state derivatives from the current state values. However the real power of Modelica comes from its acausal approach to modeling – models of components can be easily combined and the appropriate equations are automatically generated. Today, there is large library of component models written in Modelica, from resistors to electro-hydraulic devices. The Modelica community is overseen by the Modelica Association [9], a number of books and online tutorials are available, and there are regular conferences. Several proprietary modeling and simulation environments are based on the Modelica language, but there is also an open-source branch called OpenModelica. The OpenModelica compiler translates Modelica models to C code for execution.

An alternative approach is to literally draw the block diagram using a computer – a graphical user interface allows the user to add blocks and wires to a graphical canvas.

```
1 λ = 0.3
2 x = bd.INTEGRATOR(x0=1) # integrator block
3 y = bd.INTEGRATOR(x0=1) # integrator block
4 f = bd.FUNCTION(lambda x, y: -x + λ * (1 - x
  **2) * y, nin=2)
5 bd.connect(x, f[0]) # wire from output of
  integrator x
6 bd.connect(y, x, f[1]) # wires from output
  of integrator y
7 bd.connect(f, y) # wire from output of
  function block
```

Listing 3 `bdsim` model of a van der Pol oscillator, key lines from `examples/vanderpol.py`. Note that in Python, `lambda` is a reserved word (for inline function definition) but variable names may contain unicode characters.

One of the earliest such tools was MATRIXx[®] published by Integrated Systems Inc. [10] in 1983. It had a simple but effective keyboard and graphical interface (System Builder) that allowed blocks to be placed and wired, and then the resulting model could be simulated. Simulink[®], published by the MathWorks[®] in 1984, had a more sophisticated graphical interface. Importantly, Simulink was tightly integrated with MATLAB[®] which had powerful graphics and visualization capabilities as well as a growing collection of Toolboxes providing domain-specific functionality – it has gone on to become dominant in the field. Today there are few alternatives, either commercial or open-source. MATRIXx, now owned by National Instruments, is at end-of-life and no longer supported. LabVIEW published by National Instruments, another proprietary tool, provides a graphical block-diagram editor and simulation capability. OpenModelica provides the OMEdit graphical block-diagram editor. Xcos [11] is a mature open-source graphical modeling and simulation tool leveraging the Scilab language [12] and now supports Modelica integration. Collimator [13] is a recent commercial cloud-based product with a limited free tier for students.

Python, see “Introduction”, is amongst the most popular programming languages today [14]. It is open-source and has a rich ecosystem of open-source packages and development tools. Yet, despite its ubiquity there are few open-source Python tools concerned with control engineering and block diagrams, and many that do exist are incomplete or abandoned. One notable exception is the `python-control` package [15] which can perform common operations for the analysis and design of feedback control systems in support of the textbook by Åström and Murray [16]. OpenModelica includes a Python interface that allows users to access the modeling and simulation capabilities of OpenModelica from Python, but this requires the user to have knowledge of both Python and Modelica, and learn the OpenModelica tool chain. OpenModelica supports Windows and Linux but for MacOS it is only available as a docker container or it must be built from source.

The goal of **bdsim** is to bring block-diagram simulation capability to Python in a convenient and straightforward way. The approach is similar to a system-modeling language, like Simnon or Modelica. However, instead of creating a new language from scratch, **bdsim** embeds the description of the blocks and wires into a Python program by utilizing inherent features of the language. This is illustrated in Listing 3, where lines 2 and 3 create references to integrator blocks, and line 4 creates a 2-input 1-output function block. Lines 5-6 connect the integrator outputs to the inputs of the function block, and lines 6-7 connect the integrator inputs. This listing makes the integrator blocks explicit, unlike the differential-equation structure of

Listings 1 and 2.

The key features of **bdsim** are:

- » A block diagram can be expressed quickly and easily using Python code. This allows the user to leverage standard integrated development environments (IDE), standard version control tools, and standard software development workflows including continuous integration (CI).
- » Wires can communicate *any* Python type such as: `int`, `float`, `list`, `dict`, `string`, functions and objects. Vectors and multi-dimensional arrays are easily supported as NumPy [17] objects. Python dictionaries can be used to hold an heterogeneous collection of objects (somewhat like a Simulink bus): the `DICT` block turns its input signals into a multi-item `dict`, while the `ITEM` block selects the specified item from the input `dict`. All objects are passed by reference over the wires which is highly efficient for memory-consuming objects such as large matrices or images.
- » Currently over 70 blocks have been implemented, including: linear and nonlinear functions, signal sources, graphical display blocks, as well as continuous- and discrete-time dynamics. It is easy to add your own block, it is simply a class.
- » **bdsim** blocks from other Python toolboxes can be automatically discovered and included in a block diagram. In this way **bdsim** integrates with the Python Robotics ecosystem [18] that includes:
 - the Spatial Maths Toolbox [19] provides objects that represent rigid-body transformations as $SE(3)$ matrices or $se(3)$ twists, orientations as unit quaternions, angle sequences or angle-axis pairs which can all be transmitted over **bdsim** wires, and also blocks to operate on these types.
 - the Robotics Toolbox for Python [20] provides **bdsim** blocks for robot manipulator kinematics, differential kinematics and dynamics; trajectory generation; ground and aerial robot motion models and planners.
 - the Machine Vision Toolbox for Python [21] provides **bdsim** blocks to model camera projections and image Jacobians.
- » Subsystems can be independently instantiated multiple times in a system, and can be arbitrarily nested.
- » The diagram can be executed on a small low-cost computer like a RaspberryPi, or potentially even an embedded micro-controller using MicroPython [22], for real-time control.
- » An experimental Python-based graphical editor, see Fig. 1, has been developed. Models are saved as JSON (JavaScript Object Notation) files which be parsed to form a **bdsim** model that can be executed. **bdsim** can be installed simply using the command

```

1  #!/usr/bin/getenv python
2  import bdsim
3
4  sim = bdsim.BDSim()      # create simulator
5  bd = sim.blockdiagram() # create block diagram
6
7  # define the blocks
8  demand = bd.STEP(T=1, name="demand")
9  sum = bd.SUM("+")
10 gain = bd.GAIN(10)
11 plant = bd.LTI_SISO(0.5, [2, 1], name="plant")
12 scope = bd.SCOPE(styles=["k", "r--"])
13
14 # create wires between the blocks
15 bd.connect(demand, sum[0], scope[1])
16 bd.connect(plant, sum[1])
17 bd.connect(sum, gain)
18 bd.connect(gain, plant)
19 bd.connect(plant, scope[0])
20
21 bd.compile()
22 bd.report_summary()
23
24 out = sim.run(bd, T=5) # simulate for 5s

```

Listing 4 `bdsim` code version of the block diagram sketched in Fig. 2.

```
pip install bdsim
```

and the source code is available on GitHub [1].

A SIMPLE EXAMPLE

The merits of the block diagram approach were expounded above – the challenge is to make it easy to express a block diagram in Python code so that it can be simulated and its time response visualized. We will demonstrate this with the *napkin sketch* shown in Fig. 2 which we can express concisely with `bdsim` as shown in Listing 4. The program comprises just 16 lines of executable code.

The key elements of a block diagram are, of course, the blocks, and these are defined in lines 8–12. Each block is a class instance, and references are assigned to variables whose names are consistent with the red block annotations in Fig. 2. The blocks can also have user-assigned names, see lines 8 and 11, which are used in diagnostics and as labels in plots. The `SCOPE` block at line 12 has two inputs (implicit in the length of the list of line styles) which will be plotted as a solid black line and a dashed red line respectively.

After the blocks are instantiated their input and output ports are connected in lines 15–19. In `bdsim` all wires are point to point, so a *one-to-many* connection is implemented by *many* wires, for example

```
bd.connect(source, dest1, dest2, ...)
```

creates individual wires from `source`→`dest1`, `source`→`dest2` and so on. Ports are designated using

block	nc	nd	inport	source	source type
demand@	0	0			
gain.0	0	0	0	sum.0	float64
plant	1	0	0	gain.0	float64
scope.0	0	0	0	plant	float64
			1	demand	int
sum.0	0	0	0	demand	int
			1	plant	float64
	1	0			

Initial state `x0 = [0.]`

Listing 5 Run-time output of the `bdsim` program given in Listing 4.

Python indexing notation, for example `block[2]` is port 2 (the third port, since Python uses base-0 indexing) of `block`. Whether it is an input or output port depends on context. For the `connect` method, an index on the first argument refers to an output port, while on subsequent arguments it refers to an input port. If a block has only a single input or output port then no index is required, 0 is assumed.

A group of ports can be denoted using Python slice notation, for example

```
bd.connect(source[2:5], dest[3:6])
```

will connect consecutive input and output ports: `source[2]→dest[3]`, `source[3]→dest[4]`, and `source[4]→dest[5]`. The number of wires in each slice must be consistent. A *crossover connection* could be created by connecting `source[2:5]` to `dest[5:2:-1]`.

Ports can also be referenced using Python attribute syntax. For example, the `SCOPEXY` block has two input ports and, for an instance referenced by `scope`, they can be referenced as `scope[0]` and `scope[1]`, or as `scope.x` and `scope.y`. Again, whether the names refer to input or output ports depends on context.

Line 21 assembles all the blocks and wires, checks connectivity, checks for algebraic loops, instantiates subsystems to create a flat wire list, and then builds the data structures required for simulation.

Line 22 generates the tabular report, shown in Listing 5, which shows each block in the system; the number of continuous (`nc`) and discrete-time states (`nd`); the blocks whose outputs provides its inputs; and the datatype of those inputs. The block name can be provided, as is the case for the blocks `demand` and `plant`, or auto-generated based on the block type and a unique integer for that block type. The `@` symbol denotes a source block which has no inputs.

Line 24 runs the simulation for 5 seconds. Before it starts simulating, it prints the initial value of the state vector, in this case only a continuous-time state vector. The system

What is Python?

Python is an interpreted high-level programming language that is consistently ranked as one of the most popular programming languages, and is likely to be familiar to students, engineers, researchers and hobbyists. It is portable and highly scalable, running on GPU clusters for machine learning applications, desktops, low-cost embedded computers such as Raspberry Pi, and fairly complete subsets of the language can be run on “bare metal” using packages like MicroPython [22].

Python was developed by Guido van Rossum, initially as a Christmas hobby coding project in 1989, and is named after the British TV series Monty Python’s Flying Circus. van Rossum is a Dutch mathematician and computer scientist who worked at the Centrum Wiskunde & Informatica (CWI) in the Netherlands on the ABC programming language. He currently leads a team at Microsoft focussed on improving the performance of the Python interpreter.

The first version was released in 1991, Python 2 followed in 2000 (and was discontinued in 2020 with version 2.7.18),

and Python 3 was released in 2008. Python inherits ideas from many previous programming languages, including ABC, and supports procedural, object-oriented and functional programming paradigms. It has a clear and simple syntax, dynamic typing, and garbage collection.

A key feature of Python is that it is easily extended using packages. The main repository for third-party Python software, the Python Package Index (PyPI) [23], lists over 300,000 packages that can be simply installed using the `pip` command. Many of these are open-source and hosted on GitHub or similar public repositories. For engineering work, popular packages such as NumPy [17], SciPy [24] and Matplotlib [25] together provide a powerful interactive or scripted software environment that supports linear algebra, data analysis and high-quality graphics. SymPy provides computer algebra, Jupyter provides shareable and web-browseable notebooks, and code quality is enhanced by tools for linting, formatting, unit testing and coverage reporting.

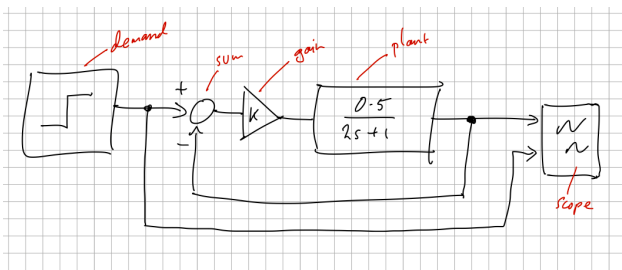


Figure 2 Hand-drawn sketch of a block diagram to be simulated. Arrow heads indicate the direction of signal flow. This example is shipped with `bdsim` as `bdsim/examples/egl.py`.

is simulated and the values of state variables and selected other signals are saved as a time series, and graphical blocks are updated, for example the `SCOPE` block shown in Fig. 3.

The simulation results are returned in a container object, essentially a *struct*,

```
>>> out
t      = ndarray:float64 (123,)
x      = ndarray:float64 (123, 1)
xnames = ['plant.x0'] (list)
ynames = [] (list)
```

which contains a 1D-array of time values `t`, a 2D-array of system-wide continuous-time state values `x` with one row per timestep and column `i` is associated with `xnames[i]`. In this case it denotes the zeroth state of the `plant` block.

Parameters can be changed after the model is defined. This allows the simulation to be run multiple times with different parameter values, perhaps for learning-based

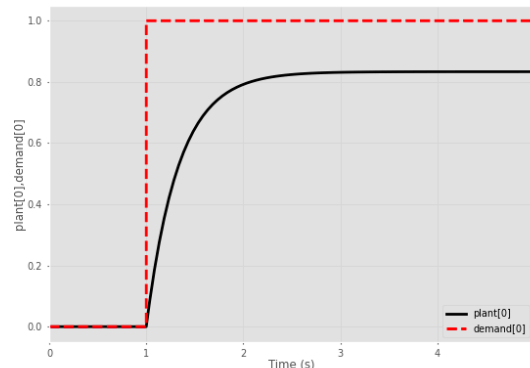


Figure 3 Scope window generated by the `run` method which performs the simulation.

control or parameter sweeps. For example, to sweep the gain of the `GAIN` block from 1 to 10 in steps of 1, we can replace line 24 of Listing 4 with

```
for K in range(1, 11):
    gain.set_param("K", K)
    out = sim.run(bd, T=5)
```

where the `set_param()` method is used to change the value of a block parameter, in this case the gain of the `GAIN` block.

To include extra signals in the simulation results object we don’t have to change the block diagram, we simply add them to the watch list at simulation time. For example, to include the output of the signal generator and the

summing junction we replace line 24 of Listing 4 with

```
out = sim.run(bd, T=5, watch=[demand, sum])
```

In this case, **bdsim** defaults to output port 0 of each block in the watch list. This information is saved in additional variables `y0`, `y1` etc. `ynames` is a list of the names of the watched variables, and the result now looks like

```
>>> out
t      = ndarray:float64 (123,)
x      = ndarray:float64 (123, 1)
xnames = ['plantx0'] (list)
y0     = ndarray:int64 (123,)
y1     = ndarray:float64 (123,)
ynames = ['demand[0]', 'sum.0[0]'] (list)
```

The simulation is run by executing the program described above

```
$ python egl.py
```

The running program will popup new windows to display the scope graphs and, by default, pause until the windows are dismissed.

Parameters of the simulation execution are controlled with increasing precedence by: environment variables, programmatic settings and command line options. For example, we can enable scope animation which shows the signals evolving over time by

```
$ python egl.py --animation
```

or we can disable graphics and save the simulation results in a Python pickle file

```
$ python egl.py --no-graphics --out egl.pickle
```

or we can override the value of the GAIN block by

```
$ python egl.py --set gain.0:K=20
```

All of this makes it easy to script and run simulations in batch mode.

Some syntactic shortcuts

There are many tradeoffs between block diagram and pure-code representation. Creating and wiring blocks is tedious, whether achieved graphically or by writing **bdsim** code. On the other hand, writing regular Python code to represent the system risks losing the clarity of the block diagram abstraction. **bdsim** supports a useful hybrid approach that allows overloaded Python operators to implicitly generate arithmetic blocks like GAIN, SUM and PROD and the required wiring. Replacing lines 7-19 of Listing 4 with

```
7 # define the blocks
8 demand = bd.STEP(T=1, name="demand")
9 plant = bd.LTI_SISO(0.5, [2, 1], name="plant")
10 scope = bd.SCOPE(styles=["k", "r--"])
11
12 # create wires between the blocks using Python
    syntax
```

```
13 scope[0] = plant
14 scope[1] = demand
15 plant[0] = 10 * (demand - plant)
```

requires 40% fewer lines of code – there are fewer blocks explicitly created and fewer lines of code required to perform the connections. Note that in lines 13–15 the left-hand side of the assignment must include an index – this is a Python syntactic requirement to distinguish a Python item or attribute assignment (implicit wire creation) from a variable assignment. Line 15 generates a “+” SUM block, a GAIN (of 10) block, and the appropriate connections.

If we generate a summary report, like Listing 5, we see that this more concise code results in *exactly the same* constituent blocks and wires. This approach strikes a nice balance between block diagram abstraction and Pythonic programming.

Using LLMs to write bdsim code

In the discussion above, we have manually transcribed the block diagram of Fig. 2 to the code shown in Listing 4. It is now possible to do this automatically using commodity AI tools. The last few years have seen significant developments in large-language models (LLMs) such as ChatGPT and Gemini, and more recently we are seeing multi-modal networks that can process both text and images.

Testing was performed on the free tier of Gemini with two hand-drawn sketches (similar in quality to Fig. 2). We excluded Fig. 2 from testing because it, and the equivalent code, are published on GitHub and the network may have trained on it.

The first prompt included a pointer to the GitHub repository and the sketch. On the first iteration Gemini showed a remarkable ability to correctly parse the diagram and accurately created the code for creating the blocks and connections. The network initially made some errors in its understanding of the factory methods for block instantiation, and the arguments to the simulation run method. It also hallucinated a non-existent alternative to the watchlist feature. After four iterations the code was perfect. The network provided default numerical values for block parameters given as algebraic symbols in the sketch, and added a signal source block. For the second, more complex sketch, Gemini created correct and runnable code on the first attempt. The code was well-structured and easy to read.

For the inverse problem, code to block diagram, the result was mixed. The first attempt gave the diagram as ASCII art but some of the blocks and wires were malformed, and some elements were duplicated. Asking for the diagram in GraphViz format produced executable GraphViz code which was a valid representation of the system. The summing junction was depicted as a circle, the other blocks were rectangles. The resulting diagram is valid but suffers the same aesthetic limitations as Fig. 6.

These limited tests show that LLMs are already a powerful way to create complex models which obviates the need to use a graphical block diagram editor. Given the rate of progress in the field, these initial results will be surpassed by the time of publication. Multimodal LLM technology will have a profound impact on control design tools going forward, and of course on control systems education.

Mixed continuous- and discrete-time system

To demonstrate this capability we can modify our example and introduce a zero-order hold (ZOH) after the gain block. This requires relatively little change (one line is removed and five lines are added).

```
# define the clocks          # ADD
clock1 = bd.clock(10, "Hz")  # ADD

# define the blocks
...
zoh = bd.ZOH(clock=clock1)   # ADD

# connect the blocks
...
bd.connect(gain, plant)    # DELETE
bd.connect(gain, zoh)        # ADD
bd.connect(zoh, plant)       # ADD
```

This results in the following changed output at execution time

block	nc	nd	inport	source	source type
demand@	0	0			
gain.0	0	0	0	sum.0	float64
plant	1	0	0	zoh.0	float64
scope.0	0	0	0	plant	float64
			1	demand	int
sum.0	0	0	0	demand	int
			1	plant	float64
zoh.0	0	1	0	gain.0	float64
	1	1			

Clocked blocks::

id	block	clock	period	offset
5	zoh.0	clock.0	0.1	0

Initial state x0 = [0.]

Initial dstate (clock.0) x0 = [0.]

where a table of clocked blocks is displayed, along with the discrete-time state associated with each clock. After running the simulation, the results are

```
>>> out
clock0::
      t = ndarray:float64 (50,)
      x = ndarray:float64 (50, 1)
t      = ndarray:float64 (173,)
x      = ndarray:float64 (173, 1)
xnames = ['plantx0'] (list)
y0      = ndarray:int64 (173,)
```

```
y1      = ndarray:float64 (173,)
ynames = ['demand[0]', 'sum.0[0]'] (list)
```

which includes a nested structure associated with the clock that contains its discrete-time state history. The system may include multiple clocks, and the discrete-time state history is associated with the relevant clock object.

Debugging

Errors such as unconnected inputs and algebraic loops (where a block's output is fed back to its input) are flagged at the compilation stage. Run-time errors typically result from invalid values on wires which are state or time dependent, and these are harder to debug. By default, at every time step all wires are checked for invalid values (NaN or infinity) and will stop the simulation. If exceptions are thrown at run time they are caught and displayed with context to indicate the time step, the block which threw it, and the inputs to the block. Dynamic blocks are a little more complex because they contain several methods that are invoked at each time step: state initialization, output computation, and derivative computation. Run-time errors can also occur at compilation stage, since that stage executes the entire diagram for one time step to determine the types of the signals on each wire.

bdsim includes a simple keyboard-based debugger, invocable by a command line option, that allows block outputs to be examined, breakpoints to be set for particular times, single stepping, and dropping into the Python debugger. A standard IDE such as Visual Studio Code can also be used to set breakpoints in user code to debug errors.

UNDER THE HOOD

Each block is defined by a class that inherits, according to its type, from a core abstract **bdsim** class: **SourceBlock**, **SinkBlock**, **FunctionBlock**, and **TransferBlock**. Each implementation provides specific methods depending on the block's type. Blocks in the diagram are instantiated by methods, such as **STEP**, of the **BlockDiagram** object, which is itself created in line 4 of Listing 4. These block methods, by convention with upper-case names, are factory methods and create instances of the block classes without having to explicitly call their constructor and connect them to their owning block diagram. These methods are *dynamically added* to the **BlockDiagram** object when it is created, based on definitions of classes discovered at startup time, line 3, from files along the block path. This allows easy extension of the core set of blocks, for example, the Spatial Maths Toolbox [19], the Robotics Toolbox for Python [26], [20] and the Machine Vision Toolbox for Python [21] provide domain specific blocks that are defined in their **blocks** folders.

Each continuous-time block has methods that return

$$\dot{x}^i = f^i(x^i, u^i, t) \quad (1)$$

$$y^i = h^i(x^i, u^i, t) \quad (2)$$

where i is the block index, x^i is a subset of the system's continuous-time state vector, u^i is the input to the block. For a static block (1) is omitted, and for a dynamic block (2) cannot depend on u^i . The continuous-time dynamics of the entire system is described by the set of ordinary differential equations

$$\dot{x} = f(x, t) \quad (3)$$

where x is the continuous state vector for the entire system and t is time. Evaluating this function requires the evaluation of the functions f^i and h^i for each block in the system. The compilation step sets the groundwork for this – it converts the hierarchical block structure to a “flat network”, and creates a dataflow schedule which ensures that blocks are executed in order such that their inputs u^i have already been computed by other blocks.

The time response $x(t)$ is obtained by integrating (3) from the initial conditions. Currently the SciPy integrators [27] are used, and the variable-step RK45 solver is the default.

Naively integrating the dynamic system of Listing 4 with a variable-step-size integrator will yield erroneous results since the integrator *steps over* the demand change at time $t = 1$. A simple solution is to enforce small fixed integration-time steps but this is inefficient and leads to increased solution time. More sophisticated integrators implement state- or time-dependent events to properly handle input discontinuities in inputs or states. Internally, **bdsim** implements an event queue, and at simulation startup each block declares event times. In this case the STEP block declares an event at $t = 1$, and the simulator integrates over the intervals $[0, 1]$ and $[1, 5]$. The event mechanism currently does not consider events which are a function of state – precluding, for example, solution of the classic bouncing ball problem.

The event mechanism is also used for discrete-time blocks, where the clocks declares an event at each time step. For each clock $c \in 1 \cdots n_c$, where n_c is the number of clocks, the compilation step builds datastructure to compute the discrete-time analog of (3)

$$x_{k+1}^c = g^c(x_k^c, t) \quad (4)$$

where x_k^c is the discrete-time state vector for clock c at time step k . The output of discrete-time blocks have their outputs held constant and the continuous-time dynamics are integrated until the next sample time, and the discrete-time states are updated and saved in a datastructure associated with the clock object.

REAL-TIME EMBEDDED CONTROL

Laboratory-scale electro-mechanical apparatus are a staple of control education, with such well known examples as 1D or 2D ball balancing, inverted pendulum, Furuta pendulum, or reaction wheel pendulum [28]. There is growing interest in low-cost apparatus based on Raspberry Pi hardware and simple hobby-class mechatronic components [29], [30]. Currently MATLAB cannot run on Raspberry Pi class hardware but Simulink models can be compiled for, exported to, and run on that platform. Telemetry allows real-time data to be displayed on the Simulink host and control parameters can be adjusted on the fly.

To perform real-time control with **bdsim** we need to add input and output blocks to the model that interface respectively with the sensors and actuators of the physical plant. A hypothetical example, based on Listing 4 would be (two lines are removed, and two are added.)

```
1 # define the blocks
2 demand = bd.STEP(T=1, name="demand")
3 sum = bd.SUM("+")
4 gain = bd.GAIN(10)
5 dac = bd.DAC(channel=0)
6 plant = bd.LTI_SISO(0.5, [2, 1], name="plant")
7     # DELETE
8 plant = bd.ADC(channel=0)
9     # ADD
10 scope = bd.SCOPE(styles=["k", "r--"])
11
12 # connect the blocks
13 scope[0] = plant
14 scope[1] = demand
15 plant[0] = 10 * (demand - plant)
16     # DELETE
17 dac[0] = 10 * (demand - plant)
18     # ADD
```

The DAC and ADC blocks are essentially “drivers” that are specific to the particular input/output hardware available on the computer and the operating system.

To evaluate the efficacy of **bdsim** for such real-time applications we have applied it to the laboratory-scale electromechanical system shown in Fig. 4. This is a resistive trackpad actuated by roll- and pitch-angle servos and the goal is to guide a metal ball rolling across the trackpad along a desired path. The **bdsim** code ran on a Raspberry Pi 4 and communicated using SPI with a Teensy 2.0 board interfaced to the hardware: reading the analog voltages from the trackpad, and generating pulse-width modulation (PWM) command signals for the servo motors. A custom **bdsim** source block outputs the ball position, and a custom sink block takes the roll- and pitch-angles as inputs. A PID controller, implemented in **bdsim**, was used for each degree of freedom, and results are shown in Fig. 5 – the oscillation is attributed to a deadband in the motor speed response. In practice we found control up to 50 Hz was possible with zero overruns and acceptable levels

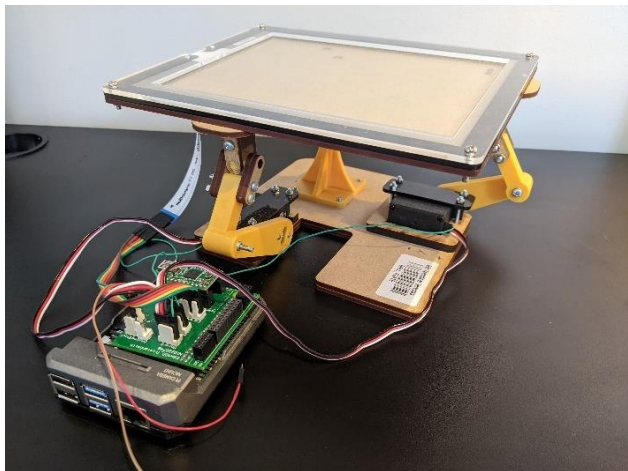


Figure 4 Experimental apparatus showing, in the foreground, the RaspberryPi 4 with the Teensy 2.0 board that interfaces to the trackpad and servo motors (image courtesy of Tristan Smith).

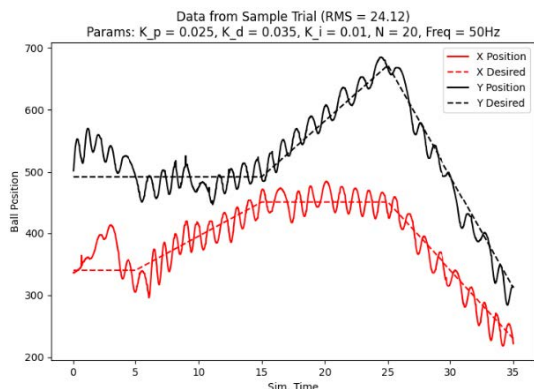


Figure 5 Control results (courtesy of Tristan Smith).

of timing jitter, despite relatively little attention given to code optimization. Future work includes network-based telemetry and remote parameter setting.

Increasing the sample rate would require tackling some bigger challenges. The RaspberryPi operating system is a Linux variant whose process scheduler is optimized for time sharing not real-time deadlines. Various kernel patches are available for Linux, or an alternative real-time operating system could be used [31]. The Python language itself is problematic for real-time control due to the non-deterministic nature of its default garbage collection (GC) process, and the global interpreter lock (GIL) which ensures the integrity of key Python interpreter data structures at the expense of thread-switching performance. Fortunately, Python development is moving in the right direction, Python 3.13 includes a just-in-time (JIT) compiler [32] and has experimental support for disabling the GIL [33].

For still higher performance there are three options. The

first is to have **bdsim** run on embedded hardware using CircuitPython or MicroPython [22] without an operating system. To achieve this requires the code to run without any operating system services, filesystem or graphics display. **bdsim** has strong reliance on NumPy and Matplotlib which are not available in this environment. NumPy could be replaced by `ulab` [34], and graphical display could be handled by a network-based telemetry system with remote graphics.

The second option is to use an Arduino or similar small computer as an input/output server, and a library like Firmata [35] or Telemetry [36] to manage communications between the host running **bdsim** and the Arduino. These protocols are quite general purpose but have some limitations for control applications which arduino [37] was designed to overcome.

The third option is to transpile the **bdsim** Python model to C code for cross-compilation to the target platform. Simulink, OpenModelica, and Xcos all have a mature tool chain for converting their respective modeling language to C or C++ code. A number of experimental transpilers convert Python to C or C++ and prominent among these are Cython [38], Nuitka [39], and Shed Skin [40]. They may offer a pathway to code generation for **bdsim**, but this is future work.

GRAPHICAL EDITING ENVIRONMENTS

bdsim provides a convenient way to encode a block diagram, but for pedagogical purposes there is a need to depict the corresponding graphical block diagram. While a short **bdsim** program like Listing 4 can be easily understood, the same is not true of the **bdsim** code associated with Fig. 1 which has 22 blocks and 23 wires. A picture is indeed worth a thousand words.

The corresponding diagram could be drawn using any computer-based drawing tool but this is somewhat tedious. Of greater concern is that it we cannot prove that the diagram we have drawn is equivalent to the **bdsim** code. Evolution of the model would require consistent updates to the drawing and the code. This, in retrospect, is the greatest advantage of a graphics-based approach like Simulink where there is a single source of truth which is held in the block diagram.

An alternative to the graphics-based approach is to automatically generate a graphical block diagram from **bdsim**'s internal lists of blocks and wires. This avoids the multiple source of truth problem, since the true representation is the **bdsim** program. A block diagram is a mathematical graph and there are several standard file formats for storing graphs, including GraphViz or GraphML, as well as tools to display them. For the system of Listing 4, a mathematical graph displayed using GraphViz [41] is shown in Fig. 6 and it provides a useful depiction of the signal flow within the system. However, it bears little

similarity to a conventional block-diagram and attempts to coerce the tool to produce such a diagram were unsuccessful. The user could add hints like block coordinates and block orientation, but this would be onerous and time consuming to finesse. There is much literature concerning the automatic layout of graphs, but no easy to use open-source packages were found that could be adapted to this task.

This means that the only workable approach is a graphical block diagram editor connected to a simulation engine, as exemplified by Simulink, LabVIEW, Xcos and OMEdit. An ideal starting point would be an open-source drawing tool, that in addition to representing blocks and wires, would support entering the parameters of the **bdsim** blocks. No such tool was found, so **bdedit** was created from scratch, initially as a student project.

bdedit is a proof-of-concept graphical tool to create, edit, render and execute block diagram models, and create high-quality graphics for inclusion in publications. It uses PyQt5 to achieve a multi-platform graphical user interface. Models are stored in a JSON file [42] with extension `.bd`. From the examples folder

```
$ bdedit eg1.bd
```

will create the user interface shown in Fig. 7. The left-hand panel is the menu of all dynamically loaded blocks organized by category – clicking a block will add it to the middle of the canvas from where it can be dragged into position. All blocks are rectangular and have input ports on the left edge and output ports on the right edge, but blocks can be flipped left to right. Right clicking a block opens a panel on the right which allows block parameters to be edited. Left clicking a block allows it to be moved on the diagram.

Wires are created by clicking and dragging from an output port to an input port. Wires adjust automatically as blocks are moved using simple layout heuristics, and grid snapping is implemented.

The JSON file describes the graphical layout parameters of the blocks and wires, as well as connectivity and block parameters. The file is human readable and editable, and could be autogenerated. The command line tool **bdrun** takes such a JSON file as an argument, parses it to form an executable block diagram using **bdsim** and executes it

```
$ bdrun eg1.bd
```

When using **bdedit**, the run button at the top left (triangle in circle) saves the current diagram as a JSON file and spawns **bdrun** as a subprocess to execute it.

Simulink enjoys a tight coupling with the interactive MATLAB environment via the MATLAB *workspace*. Simulink results can be saved to the MATLAB workspace, and MATLAB workspace variables can be used as block parameters. **bdedit** has no *workspace* and block param-

eters are limited to values that can be entered by means of **bdedit** and be represented as a string in a JSON file. While this is fine for constants such as numbers, strings, and lists of those things, it cannot support the creation of custom objects like NumPy arrays, robot kinematic models or camera models.

To overcome this limitation we allow the parameter value to be expressed as “=var” which means that the value of the variable `var` is determined at run time – essentially a very late binding. This means that before the model defined by the JSON file can be run, we need to create the variables that it references. This is achieved by a `main` block which has a single parameter that is the name of a Python module. When a model with a `main` block is run from **bdedit** that module is spawned and it is passed, using command line arguments, the JSON file describing the block diagram. The “main” module is specific Python user code that creates the variables required, parses the JSON file to build a block diagram, and then executes it using **bdrun**.

bdedit includes a number of simple features to improve the readability of large and complex diagrams as shown in Fig. 1. Grouping boxes, coloured rectangles behind the blocks are a powerful way to add structure to a model. Additional text can be added to the diagram to label wires and grouping boxes.

ADVANTAGES, LIMITATIONS AND FUTURE WORK

bdsim, like all tools, has strengths and weaknesses. **bdsim** follows the Unix philosophy [43] of doing one thing well, and that thing is simulation. It uses plain text input and output, is modular and can be readily combined with other tools by coding or scripting. It does not perform system analysis or support a model-based design workflow, and in that respect is certainly not a replacement for monolithic tools like Simulink, Modelica, or LabVIEW.

For smaller projects in teaching and research it is very capable, particularly those that involve complex system of systems such as shown in Fig. 1 and/or require tight integration with Python code. The impetus for **bdsim**’s creation was writing the textbook [3] with example code provided in Python. That book is based on an earlier edition [44] which has example code in MATLAB and over thirty Simulink models. Translating the MATLAB examples to Python was straightforward, but the Simulink models posed a challenge. The solution, **bdsim**, has a subset of Simulink’s capability but more than enough capability to run those thirty models, such as Fig. 1, to help demonstrate the principles of robotics, vision and control required by the book.

Python is a general-purpose language that is easy to learn and use, and is increasingly the language of choice for undergraduate engineering education. Python

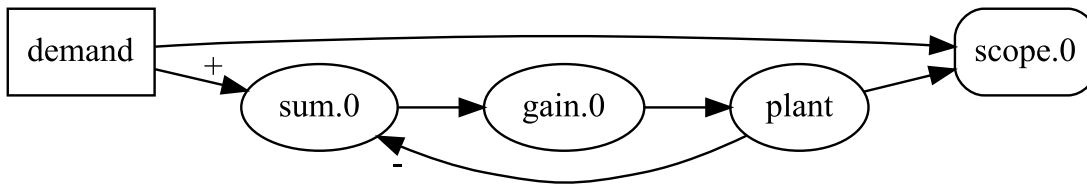


Figure 6 A graph representation of the model of Listing 4. The GraphViz format graph was created by the blockdiagram's `dotfile` method and processed by the GraphViz `dot` utility. Signal sources are forced to the left and sinks to the right.

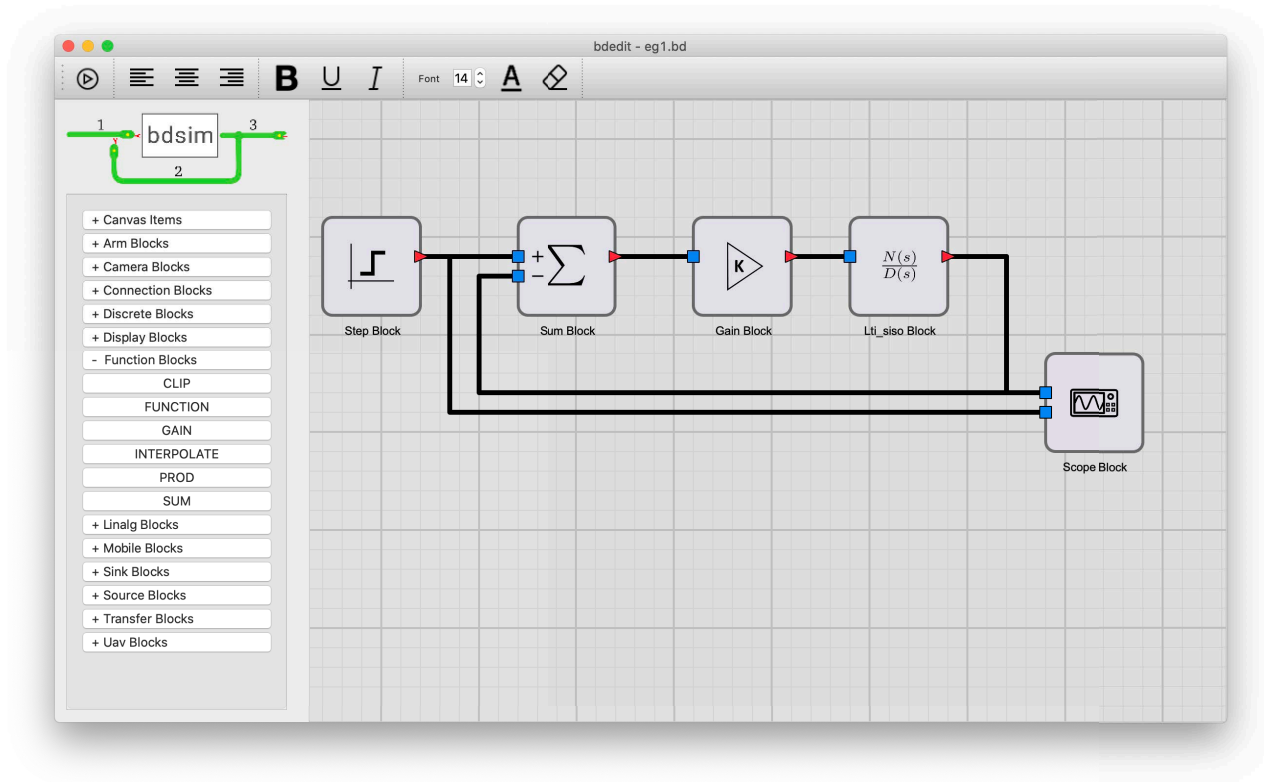


Figure 7 Main window of the `bdedit` block diagram editor showing the same system as Fig. 2. The visual style deliberately avoids mimicing well known proprietary tools to implicitly reset user expectations.

is the number one language in the TIOBE index [14] and is widely used in industry particularly in the fields of robotics, computer vision and AI. Python runs under all major computer operating systems and even micro-controllers [22]. It can run on cloud servers, on a desktop, on small and cheap RaspberryPi-like computers, or even in a web browser using web assembly language (WASM). With suitable input/output modules it can also be used to create models that run in real-time.

`bdsim` is open-source and free, but it is not supported.

It is a one-person passion project whereas the commercial tools have large teams of developers and testers.

There are many directions in which this package could be extended and improved. The simplest improvements are to simply extend the repertoire of blocks. The event queue is a cumbersome workaround for the limitations of the SciPy numerical integrator and should be reengineered. The addition of state-dependent events would allow simulation of hybrid systems. The block dataflow execution scheduler is rather primitive and inefficient – continuous-

time static blocks that feed only clocked blocks need only be evaluated at the clock instant, not at every integrator time step as currently occurs. Extending the underlying scheduler to utilize multiple cores could improve simulation time. Currently algebraic loops are not supported.

For real-time control the various options discussed above should be further investigated: Python without the GIL running on real-time enabled Linux; embedded Python; and Python to C code transpilation. The **bdsim** block library should include blocks for interfacing with digital and analog input/output lines and supporting the huge variety of input/output technologies available. Work could be done to better support headless embedded systems by networked parameter setting and remote graphical display of telemetry data.

It must be stressed that the graphical editor **bdedit** is a proof of concept and needs further work – it is fragile and prone to occasional crashes. It may be best to reimplement it using modern browser-based technologies which have portability advantages over the traditional PyQt-style desktop windowing interface. Alternatively, autogeneration of a graphical representation from **bdsim**'s block and wire lists could be pursued.

CONCLUSION

This article has introduced an open-source Python package **bdsim** that enables modeling and simulation of continuous- and/or discrete-time dynamic systems. Systems are conceptualized in block diagram form, but expressed as Python code – blocks are represented by Python objects. The design approach and usage has been explained using detailed examples which have included a real-time variant.

Creating a block-diagram model by coding is quick and efficient, even for large systems. The approach allows leverage of mature software engineering tools and methodologies, and the resulting model can be run on embedded platforms or computing clusters. However, there is a human desire to see the model in block-diagram form and turning a code representation back into a block-diagram is challenging. Maintaining separate representations, a drawing and a code model, is fraught with danger. For this reason there is merit in using a graphical editor to create the diagram, and using that diagram as a single point of truth. An experimental graphical editor has been introduced. In the future LLMs may obviate the need for traditional graphical entry, directly transforming a block diagram sketch to runnable code.

Further details, and current status, of the **bdsim** project can be found on the project's GitHub page [1]. Contributions and collaborations are very welcome.

ACKNOWLEDGMENT

This work has been supported by the Queensland University of Technology (QUT) Centre for Robotics. The author would like to thank and acknowledge a number of former QUT students who have helped with road testing **bdsim** (Callum Hays and Tristan Smith), and developing the graphical editor **bdedit** (Daniel Petkov, Rory Higgins, Samara Barwick, and John Wishart).

AUTHOR INFORMATION

Peter Corke (peter.corke@ieee.org) received the B.Eng. and M.Eng.Sc. in Electrical Engineering, and PhD in Mechanical and Manufacturing Engineering – all from the University of Melbourne. He is a distinguished professor emeritus at Queensland University of Technology (QUT), and founding director of the QUT Centre for Robotics. He created widely used open-source Python and MATLAB software for teaching and research; wrote the best-selling textbook “Robotics, Vision, and Control”, currently in its third edition; created the online Robot Academy; and has won national and international recognition for teaching including: the 2025 Joseph F. Engelberger Award for education; 2017 Australian University Teacher of the Year; and 2015 QS-Wharton Engineering & IT gold award. He is a fellow of the IEEE, the Australian Academy of Technology and Engineering, the Australian Academy of Science; advisory board member for IEEE Trans. Robotics Practice and IEEE Trans. Field Robotics; former editor-in-chief of the IEEE Robotics & Automation magazine; founding editor of the Journal of Field Robotics; founding multi-media editor of the International Journal of Robotics Research; holds a number of patents; and has held visiting positions at Oxford, University of Illinois, Carnegie-Mellon University, and University of Pennsylvania.

References

- [1] P. Corke, “bdsim: Block diagram simulator,” 2020, [Online; accessed 12-02-2023]. [Online]. Available: <https://github.com/petercorke/bdsim>
- [2] “bdsim on pypi,” [Online; accessed 12-02-2023]. [Online]. Available: <https://pypi.org/project/bdsim>
- [3] P. Corke, *Robotics, Vision and Control: Fundamental Algorithms in Python*, 3rd ed., ser. Springer Tracts in Advanced Robotics. Springer, 2023.
- [4] G. F. Franklin, J. D. Powell, and A. E. Naeini, *Feedback Control of Dynamic Systems*, 8th ed. Pearson, 2019.
- [5] H. Elmqvist, *SIMNON - User's Manual*, ser. Research Reports TFRT-3091. Department of Automatic Control, Lund Institute of Technology (LTH), 1975.
- [6] “Van der pol oscillator,” 2025, [Online; accessed 14-03-2025]. [Online]. Available: <http://omwebbook.openmodelica.org/static/Variables/VanDerPol.html>
- [7] M. Association, “Modelica – A Unified Object-Oriented Language for Systems Modeling. Language Specification version 3.7-dev,” 2025, [Online; accessed 14-03-2025]. [Online]. Available: <https://specification.modelica.org/master/MLS.pdf>
- [8] M. M. Tiller, “Modelica by Example,” 2019, [Online; accessed 14-03-2025]. [Online]. Available: <https://mbe.modelica.university>
- [9] “Modelica association,” 2021, [Online; accessed 02-04-2023]. [Online]. Available: <https://modelica.org>

- [10] "Integrated Systems Inc." 2020, [Online; accessed 12-02-2023]. [Online]. Available: https://en.wikipedia.org/wiki/Integrated_Systems_Inc.
- [11] "Xcos dynamic systems modeler, simulator in discrete and graphical editor," 2023, [Online; accessed 02-04-2023]. [Online]. Available: <https://www.scilab.org/software/xcos>
- [12] "SciLab," 2023, [Online; accessed 02-04-2023]. [Online]. Available: <https://www.scilab.org>
- [13] "Collimator – engineering tool for data driven modeling and simulation of dynamical systems," 2022, [Online; accessed 02-04-2023]. [Online]. Available: <https://www.collimator.ai>
- [14] "Tiobe index," 2025, [Online; accessed 14-03-2025]. [Online]. Available: <https://www.tiobe.com/tiobe-index/>
- [15] S. B. Fuller, B. Greiner, J. Moore, R. Murray, R. van Paassen, and R. Yorke, "The Python Control Systems Library (python-control)," in *2021 60th IEEE Conference on Decision and Control (CDC)*, Austin, TX, USA, December 14-17, 2021. IEEE, 2021, pp. 4875–4881. [Online]. Available: <https://doi.org/10.1109/CDC45484.2021.9683368>
- [16] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, 2nd ed. USA: Princeton University Press, 2021.
- [17] "Numpy," [Online; accessed 02-04-2023]. [Online]. Available: <https://numpy.org>
- [18] J. Haviland and P. Corke, *Python Robotics*, 2022. [Online]. Available: <https://pythonrobotics.io>
- [19] P. Corke, "Spatial Maths Toolbox for Python," 2020, [Online; accessed 12-02-2023]. [Online]. Available: <https://github.com/petercorke/spatialmath-toolbox-python>
- [20] P. Corke and J. Haviland, "Robotics Toolbox for Python," 2020, [Online; accessed 12-02-2023]. [Online]. Available: <https://github.com/petercorke/robotics-toolbox-python>
- [21] P. Corke, "Machine Vision Toolbox for Python," 2020, [Online; accessed 12-02-2023]. [Online]. Available: <https://github.com/petercorke/machinevision-toolbox-python>
- [22] "Micropython," [Online; accessed 12-02-2023]. [Online]. Available: <https://micropython.org>
- [23] "Python package index," [Online; accessed 12-02-2023]. [Online]. Available: <https://pypi.org>
- [24] "Scipy," [Online; accessed 02-04-2023]. [Online]. Available: <https://www.scipy.org>
- [25] "Matplotlib," [Online; accessed 02-04-2023]. [Online]. Available: <https://matplotlib.org>
- [26] P. Corke and J. Haviland, "Not your grandmother's toolbox – the Robotics Toolbox reinvented for Python," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11 357–11 363.
- [27] "Scipy integrators," 2025, [Online; accessed 14-03-2025]. [Online]. Available: <https://docs.scipy.org/doc/scipy/reference/integrate.html>
- [28] M. W. Spong, P. Corke, and R. Lozano, "Nonlinear control of the reaction wheel pendulum," *Autom.*, vol. 37, no. 11, pp. 1845–1851, 2001. [Online]. Available: [https://doi.org/10.1016/S0005-1098\(01\)00145-5](https://doi.org/10.1016/S0005-1098(01)00145-5)
- [29] R. M. Reck and R. S. Sreenivas, "Developing an Affordable and Portable Control Systems Laboratory Kit with a Raspberry Pi," *Electronics*, vol. 5, no. 3, 2016. [Online]. Available: <https://www.mdpi.com/2079-9292/5/3/36>
- [30] Á. Hoyo, J. L. Guzmán, J. C. Moreno, and M. Berenguel, "Teaching Control Engineering Concepts using Open Source tools on a Raspberry Pi board," *IFAC-PapersOnLine*, vol. 48, no. 29, pp. 99–104, 2015, iFAC Workshop on Internet Based Control Education IBCE15. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896315024787>
- [31] C. Tan, "Raspberry Pi Real-Time OS (RTOS): Which to Choose?" 2021, [Online; accessed 12-02-2023]. [Online]. Available: <https://all3dp.com/2/rtos-raspberry-pi-real-time-os>
- [32] B. Bucher, "PEP 744 – jit compilation," Python Software Foundation, Tech. Rep., 2024.
- [33] S. Gross, "PEP 703 – making the global interpreter lock optional in cpython," Python Software Foundation, Tech. Rep., 2023.
- [34] "ulab – a Numpy-like array manipulation library for MicroPython and CircuitPython," 2021, [Online; accessed 02-04-2023]. [Online]. Available: <https://github.com/v923z/micropython-ulab>
- [35] "Firmata, a protocol for communicating with microcontrollers," [Online; accessed 21-10-2024]. [Online]. Available: <https://github.com/firmata/arduino>
- [36] "The Telemetrix project," [Online; accessed 02-04-2023]. [Online]. Available: <https://mryslab.github.io/telemetrix>
- [37] "arduIO, an Arduino server for real-time control," [Online; accessed 21-10-2024]. [Online]. Available: <https://github.com/petercorke/arduIO>
- [38] S. Yegulalp, "What is Cython? Python at the speed of C," *InfoWorld*, 2022, [Online; accessed 02-04-2023]. [Online]. Available: <https://www.infoworld.com/article/3250299/what-is-cython-python-at-the-speed-of-c.html>
- [39] —, "Intro to Nuitka: A better way to compile and distribute Python," *InfoWorld*, 2022, [Online; accessed 02-04-2023]. [Online]. Available: <https://www.infoworld.com/article/3673932/intro-to-nuitka-a-better-way-to-compile-and-distribute-python-applications.html>
- [40] "Shed skin documentation," 2022, [Online; accessed 02-04-2023]. [Online]. Available: <https://shedskin.readthedocs.io/en/latest/documentation.html>
- [41] "GraphViz – open source graph visualization software," [Online; accessed 02-04-2023]. [Online]. Available: <https://graphviz.org>
- [42] M. Tyson, "What is JSON? The universal data format," [Online; accessed 12-02-2023]. [Online]. Available: <https://www.infoworld.com/article/3222851/what-is-json-a-better-format-for-data-exchange.html>
- [43] "Unix philosophy," 2025, [Online; accessed 14-03-2025]. [Online]. Available: https://en.wikipedia.org/wiki/Unix_philosophy
- [44] P. Corke, *Robotics, Vision & Control: Fundamental Algorithms in MATLAB*, 2nd ed., ser. Springer Tracts in Advanced Robotics. Springer, 2017.